

Αρχιτεκτονική Systematic Trading Framework

Τελευταία ενημέρωση: 2026-06-27

Το framework είναι config-driven σύστημα για research, ML evaluation, backtesting και paper/demo execution. Η αρχιτεκτονική δίνει προτεραιότητα σε χρονική αιτιότητα, reproducibility και καθαρά package boundaries.

Υψηλού επιπέδου εικόνα

```
YAML config
|
v
config loader + validation
|
v
canonical experiment pipeline
|
+-- data loading / PIT hardening
+-- feature generation + helpers
+-- target generation
+-- model training / inference
+-- signal generation
+-- backtesting / portfolio / evaluation
+-- monitoring / execution / artifacts
```

To stable CLI entrypoint είναι:

```
python -m src.experiments.runner path/to/config.yaml
```

To canonical pipeline facade είναι:

- `src.pipelines.canonical_pipeline.run_canonical_pipeline`
- registry name: `canonical_experiment`
- implementation facade προς `src.experiments.runner.run_experiment`
- πραγματική ενορχήστρωση στο `src.experiments.orchestration.pipeline.run_experiment_pipeline`

Σειρά εκτέλεσης

1. Φόρτωση YAML.
2. Config validation.
3. Data loading.
4. Point-in-time hardening.
5. OHLCV/data contract validation.

6. Feature generation.
7. Feature helpers και output mappings.
8. Target generation, αν υπάρχει model target.
9. Model training/inference ή model pipeline stages.
10. Signal generation.
11. Backtest/evaluation.
12. Monitoring, execution output και artifact writing.

Package boundaries

src/src_data

Ιδιοκτήτης για loading, PIT hardening και data validation. Δεν πρέπει να ξέρει για models ή signals.

src/features

Ιδιοκτήτης για raw feature builders και helper transforms. Δεν πρέπει να εισάγει signals, models, experiments ή backtesting.

src/targets

Ιδιοκτήτης για label/target builders. Επιτρέπεται να κοιτάει future bars μόνο για label generation. Δεν πρέπει να εισάγει models ή backtesting.

src/models

Ιδιοκτήτης για model wrappers, fold-safe preprocessing και training adapters. Δεν φορτώνει data από μόνο του.

src/signals

Ιδιοκτήτης για decision logic πάνω σε ήδη υπάρχοντα feature/model columns. Δεν πρέπει να κάνει βαρύ feature engineering εσωτερικά.

src/experiments

Ιδιοκτήτης για orchestration, reporting, Optuna/search support και artifacts. Δεν πρέπει να φιλοξενεί canonical registries.

src/backtesting, src/risk, src/portfolio, src/evaluation

Ιδιοκτήτες για εκτέλεση backtest, sizing/costs/constraints και metrics. Τα executed trade path fields, όπως realized R, MFE/MAE, bars held και exit reason, ανήκουν στο **src/backtesting**. Τα reusable trade lifecycle summaries και counterfactual diagnostics ανήκουν στο **src/evaluation** και γράφονται από το orchestration/artifact layer. Αυτά τα diagnostics είναι reporting/EDA outputs και δεν πρέπει να χρησιμοποιούνται ως model features.

src/execution

Ιδιοκτήτης για paper/demo execution outputs και MT5 demo runner. Live/demo orders προστατεύονται από explicit safety gates.

src/market_making

Ιδιοκτήτης για event-driven market-making components: quote generation, paper engine, risk checks, diagnostics, MOMENT research dataset/model/filter helpers και experiment artifact writers. Τα research configs ανήκουν κάτω από `config/experiments/market_making/` και γράφουν outputs στο `logs/experiments/market_making/`. Τα demo/live adapters παραμένουν ξεχωριστά και δεν ενεργοποιούνται από research experiments.

Registries

Τα canonical registries βρίσκονται ανά package:

- `src/features/registry.py`
- `src/signals/registry.py`
- `src/targets/registry.py`
- `src/models/registry.py`
- `src/pipelines/registry.py`

Κάθε registry:

- ορίζεται από λίστα (`name`, `callable`) ή lazy entries,
- περνά από shared registry builder όπου χρειάζεται,
- αποτυγχάνει με informative error για άγνωστα names,
- εκθέτει public resolver όπως `get_feature_fn`, `get_signal_fn`, `get_target_builder`, `get_model_builder`.

Το παλιό `src.experiments.registry` είναι compatibility facade και δεν είναι πλέον ο ιδιοκτήτης των mappings.

Feature architecture

Τα features χωρίζονται σε τρεις κατηγορίες:

1. Raw feature builders, π.χ. `atr`, `vwap`, `roofing_filter`, `hilbert_transform`, `schaff_trend_cycle`.
2. Transform helpers, π.χ. `ratio`, `difference`, `lag`, `reciprocal`, `crossing_flag`, `threshold_flag`, `rising_flag`, `between_flag`, `rolling_mean`, `rolling_std`, `rolling_sum`, `rolling_zscore`, `rolling_clip`, `rolling_linear_regression`, `rms`, `slope`.
3. Normalization helpers, π.χ. `returns`, `atr_distances`, `volatility`, `rolling_zscores`, `rolling_percent_rank`, `robust_zscore`, `volatility_scaled_return`, `atr_scaled_distance`, `range_position`, `realized_vol_percentile`, `volume_relative`, `rolling_beta_residual`.

Raw feature modules δεν πρέπει να παράγουν derived helper columns by default. Παράγωγα columns δηλώνονται στο YAML με `transforms` ή `normalizations`.

Παράδειγμα:

```
features:
  - step: hilbert_transform
    params:
      price_col: close
```

```

window: 64
amplitude_col: hilbert_amplitude_64
phase_col: hilbert_phase_64
instantaneous_frequency_col: hilbert_frequency_64
transforms:
  reciprocal:
    params:
      source_col: hilbert_frequency_64
      use_abs: true
      output_col: hilbert_dominant_cycle_64
  between_flag:
    params:
      source_col: hilbert_dominant_cycle_64
      lower: 10.0
      upper: 48.0
      output_col: hilbert_cycle_ok_64

```

Signal architecture

Canonical signal names βρίσκονται στο `src/signals/registry.py`. Ένα signal:

- διαβάζει feature/model columns,
- παράγει signal/side/weight/probability-derived output,
- δεν κάνει data loading,
- δεν εκπαιδεύει model,
- δεν παράγει helper-style diagnostics που ανήκουν στα features.

Deprecated aliases υπάρχουν μόνο για migration παλιών configs.

Target architecture

To target dispatch γίνεται από `src/targets/registry.py`.

Canonical targets περιλαμβάνουν:

- `forward_return`
- `future_return_regression`
- `triple_barrier`
- `directional_triple_barrier`
- `r_multiple`

Κάθε target πρέπει να τεκμηριώνει horizon, output columns και χρήση future information.

Model architecture

To model resolution γίνεται από `src/models/registry.py`. Τα entries είναι lazy ώστε το import του registry να μη φορτώνει βαριές ML dependencies.

Παραδείγματα:

- `logistic_regression_clf`

- `elastic_net_clf`
- `xgboost_clf`
- `lightgbm_clf`
- `lightgbm_regressor`
- `sarimax_forecaster`
- `garch_forecaster`
- `lstm_forecaster`
- `patchtst_forecaster`
- `tft_forecaster`
- `chronos_bolt_forecaster`
- `chronos_2_forecaster`
- `timesfm_2p5_200m_forecaster`
- `timesfm_1p0_200m_forecaster`
- `ppo_agent`

Το preprocessing πρέπει να είναι train-fold safe. Scalers/encoders δεν γίνονται fit σε όλο το dataset πριν από split.

Config validation

Το `src/utils/config_validation.py` είναι το κεντρικό σημείο για YAML contract checks. Ελέγχει:

- άγνωστα blocks/keys,
- τύπους παραμέτρων,
- feature helper names,
- deprecated/unsupported feature params,
- model/signal/target schema constraints.

Το `src/utils/config.py` παραμένει stable facade για loading.

Legacy και compatibility

Δεν διαγράφονται απότομα παλιά paths όταν υπάρχει πιθανότητα να χρησιμοποιούνται από configs/scripts. Compatibility facades πρέπει να μένουν μόνο όσο χρειάζεται για migration.

Τρέχοντες legacy candidates:

- feature-stage signal compatibility entries,
- `src.experiments.registry`,
- παλιά experiment/support diagnostics,
- deprecated signal aliases,
- re-export paths για backtest/trade helpers.

Κανόνες επέκτασης

Για νέο feature/signal/target/model:

1. Πρόσθεσε module στο σωστό package.
2. Κράτα τις εξαρτήσεις εντός boundary.
3. Πρόσθεσε registry entry.

4. Πρόσθεσε config validation αν εισάγεται νέο YAML contract.
5. Πρόσθεσε tests για contract, edge cases και causality.
6. Ενημέρωσε docs/catalog.

Για cross-package refactor, πρώτα γράψε migration plan. Μην αλλάζεις silent runtime defaults χωρίς explicit τεκμηρίωση και tests.